

Gemm-矩阵分块与寄存器优化

1. Gemm概念

本文以fp16的基础实现GemmLikeConv Kernel为例，介绍Gemm的一些优化方法。

```
141 #ifdef ENABLE_ARM_FP16
142 template <>
143 void ConvCompute<PRECISION(kFP16), PRECISION(kFP16)>::PrepareForRun() {
144     PARAM_INIT
145     /// select conv impl
146     auto act_param = param.activation_param;
147     auto act_type = act_param.active_type;
148     bool has_active = act_param.has_active;
149     bool pads_less = ((paddings[1] < 2) && (paddings[3] < 2));
150     bool conv_3x3_wino = (ic <= 8) || (oc <= 8);
151     bool stride_less = (sw == 1) || (sw == 2);
152     if (param.groups == ic && ic == oc) {
153     } else if (param.groups == 1 && kw == 3 && sw == 2 && no_dilation && ...
154     } else if (param.groups == 1 && kw == 3 && sw == 1 && no_dilation && ...
155     } else {
156         impl_ = new GemmLikeConv<PRECISION(kFP16), PRECISION(kFP16)>;
157     }
```

1.1 矩阵乘定义

矩阵乘通常定义为：

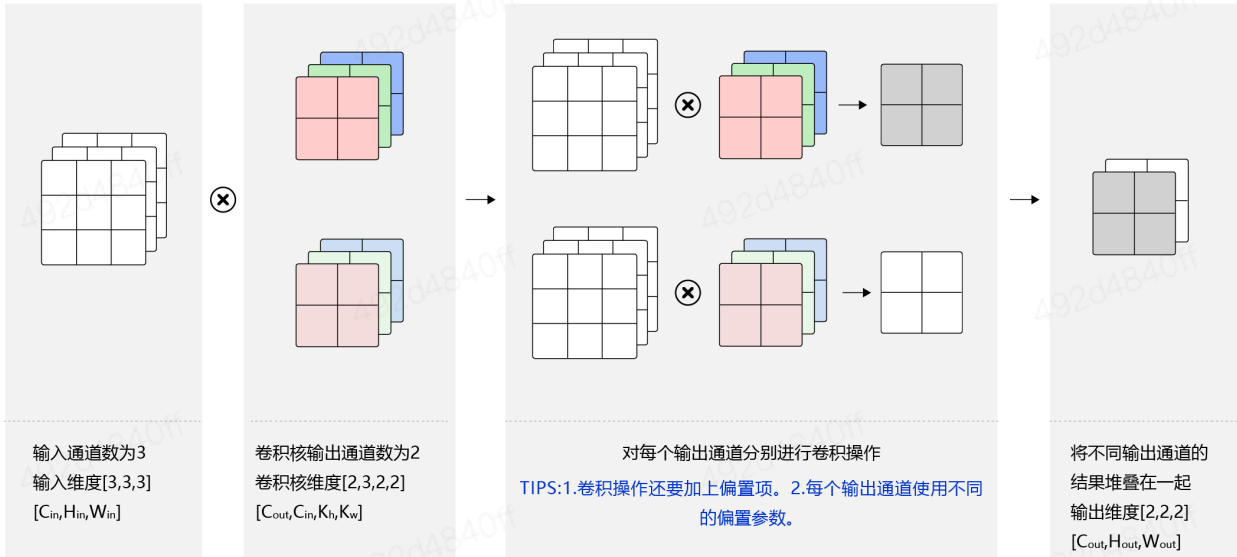
$$C = AB; A, B, C \in R^{n \times n}$$
$$C_{m,n} = \sum_{k=1}^K A_{m,k} B_{k,n}; m, n, k \in R^n$$

其中 A、B、C 三者的形状分别为 MxK 、 KxN 、 MxN，三者分别对应卷积中的weight矩阵，输入特征图矩阵以及输出特征图矩阵。

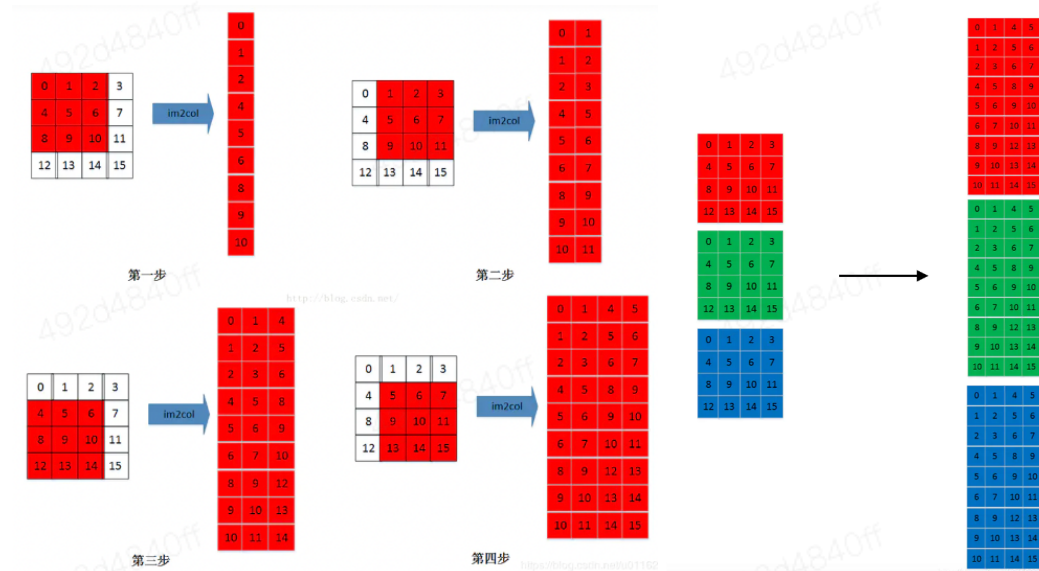
1.2 img2col---从卷积运算到矩阵乘法

通常的卷积运算如下所示：

- 输出通道的数目通常也被称作卷积核的个数，这里有两个卷积核。
- 红绿蓝代表第一个卷积核的三个输入通道；浅红浅绿浅蓝代表第二个卷积核的三个输入通道。

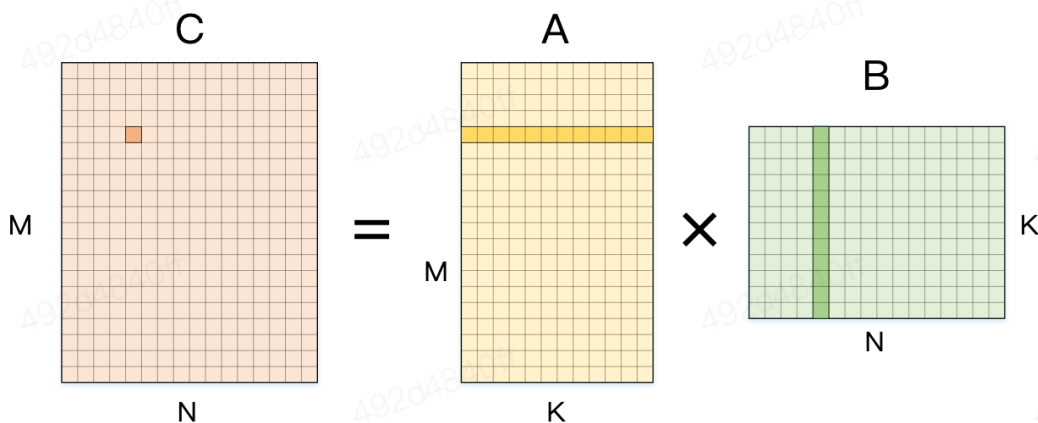


对输入特征图矩阵做如下img2col操作即可得到GEMM中的B矩阵，因此K=ic x kh x kw，N=oh x ow。



对weight矩阵而言，由于其默认为NCHW的排列，因此A中的一行数据即对应了一个产生一个oc数据所需要的weights参数，因此M=oc。

下图是计算一个输出点的矩阵乘可视化展示。



与之相对应的伪代码表示为：

</> Gemm实现伪代码

C++ | 收起 ^

```
1 for (int m = 0; m < M; m++) {
2   for (int n = 0; n < N; n++) {
3     C[m][n] = 0;
4     for (int k = 0; k < K; k++) {
5       C[m][n] += A[m][k] * B[k][n];
6     }
7   }
8 }
```

当前Lite实现主要基于软件优化的方法对这样的矩阵乘算法进行优化，核心就是基于对计算机体系机构和软件系统的特征分析，结合具体计算的特性，设计出针对性的优化方法。对矩阵乘而言，主要是选择性地调整计算顺序、改进访存局部性、利用向量指令、循环拆分向量化等优化手段。

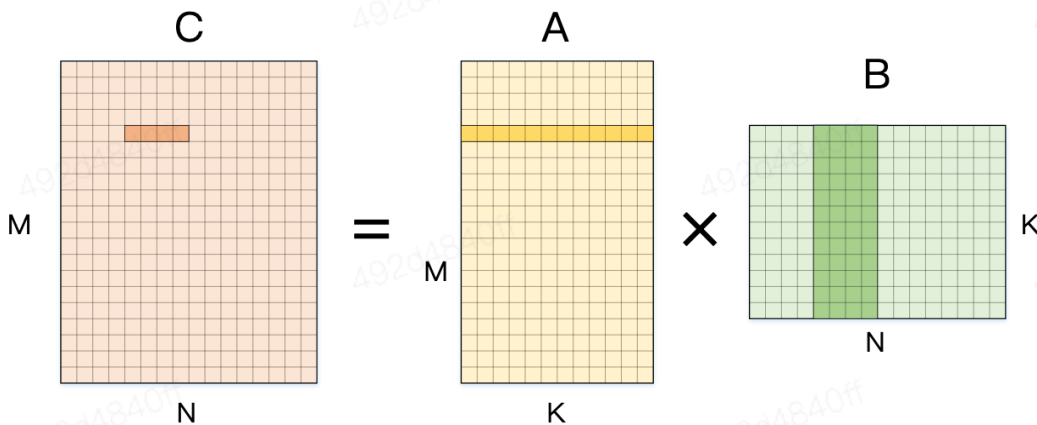
可以发现，在上述计算中乘法和加法的计算量为 $2MNK$ (计算C中每个元素时，分别需要计算 k 次加法和乘法，C的总元素个数为 MN)，访存量为 $2MNK$ (计算每个C中元素需要进行 $2K$ 次访存)+ $2MNK$ (每次计算都需要对C中的元素进行读写)。由于计算量固定(排除 Strassen算法)，所以只能优化访存，使得乘法和加法运算达到处理器的极限性能，从而实现Gemm的最佳性能。

[How to optimize gemm](#) 介绍了如何采用各种优化方法，将最基础的计算改进了约七倍。其基本方法是将输出划分为若干个 4×4 子块，以提高对输入数据的重用。同时大量使用寄存器，减少访存；向量化访存和计算；消除指针计算；重新组织内存充分利用cache。具体实现可以参考链接[repo](#)。

2. 常用优化手段

计算拆分

以上章中介绍的计算为例（浮点数据类型为float），首先将输出的计算拆分为 1×4 的小块，即对 N 维度进行拆分。计算该块输出时，需要使用A矩阵的1行以及B矩阵的4列数据。



下面是该计算的伪代码表示（外积矩阵乘算法，本文采取的矩阵乘算法）。这里已经将 1×4 中 N 维度的内部拆分进行了展开。当前内存访问所需的操作数尚未出现变化，仍然是 $4MNK$ ，但接下来会逐步改进。

</> Gemm-1x4伪代码

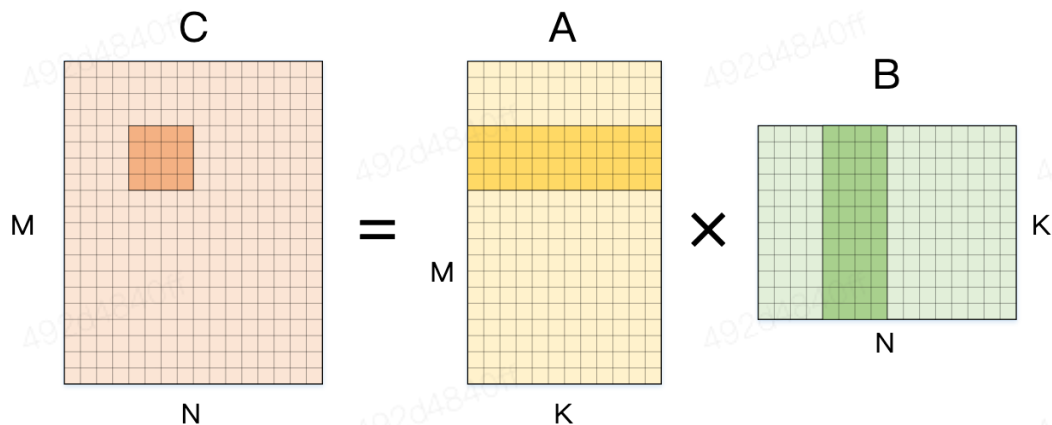
```

1 for (int m = 0; m < M; m++) {
2   for (int n = 0; n < N; n += 4) {
3     C[m][n + 0] = 0;
4     C[m][n + 1] = 0;
5     C[m][n + 2] = 0;
6     C[m][n + 3] = 0;
7     for (int k = 0; k < K; k++) {
8       C[m][n + 0] += A[m][k] * B[k][n + 0];
9       C[m][n + 1] += A[m][k] * B[k][n + 1];
10      C[m][n + 2] += A[m][k] * B[k][n + 2];
11      C[m][n + 3] += A[m][k] * B[k][n + 3];
12    }
13  }
14 }

```

通常将最内层循环称作计算核（micro kernel）。可以发现上述伪代码中，最内侧计算使用的矩阵 A 的元素是一致的。因此可以将 $A[m][k]$ 读取到寄存器中，从而实现4次数据复用。进行这样的优化后，访存量变为 $(2+1+1/4)*MNK$ ，其中1/4是对A中元素进行内存复用的效果。

类似地，如下图所示，我们可以继续拆分输出的 M 维度，从而在内侧循环中计算 4×4 输出。



将计算核展开，可以得到下面的伪代码。这里我们将 1x4 中展示过的 N 维度的计算简化表示，当前这种拆分可看成是 $4 \times 1 \times 4$ ，对 M 维度的计算进行拆分后，最内侧计算使用的矩阵 B 的元素 ($B[k][n + 0..3]$) 是一致的。因此，同样可以使用寄存器来实现4次数据复用，这样，对 A 和 B 矩阵的数据访存均可复用四次。

由于乘数效应， 4×4 的拆分可以将对输入数据的访存缩减到 $2MNK + 1/4MNK + 1/4MNK = 5/2MNK$ 。这相对于最开始的 $4MNK$ 已经得到了 1.6X 的改进，而这些改进都是通过展开循环后利用寄存器存储数据减少访存得到的。

</> Gemm-4x4伪代码

```

1 for (int m = 0; m < M; m += 4) {
2   for (int n = 0; n < N; n += 4) {
3     C[m + 0][n + 0..3] = 0;

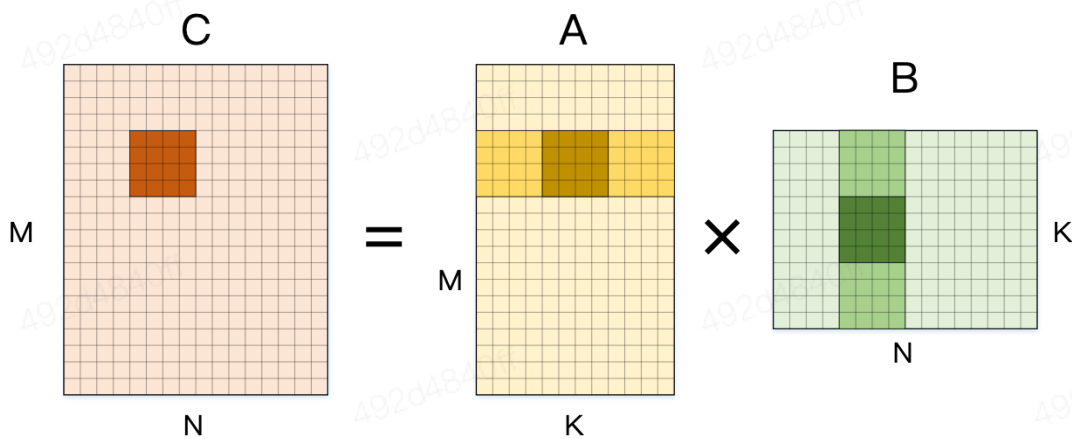
```

```

4   C[m + 1][n + 0..3] = 0;
5   C[m + 2][n + 0..3] = 0;
6   C[m + 3][n + 0..3] = 0;
7   for (int k = 0; k < K; k++) {
8       C[m + 0][n + 0..3] += A[m + 0][k] * B[k][n + 0..3];
9       C[m + 1][n + 0..3] += A[m + 1][k] * B[k][n + 0..3];
10      C[m + 2][n + 0..3] += A[m + 2][k] * B[k][n + 0..3];
11      C[m + 3][n + 0..3] += A[m + 3][k] * B[k][n + 0..3];
12  }
13  }
14 }

```

到目前为止，我们只是在输出的两个维度上进行计算拆分，而整个计算还包含最内层的规约维度 K (Reduction)。下图展示了在计算 4x4 输出时，将维度 K 拆分，从而每次最内层循环计算出输出矩阵 C 的 4/K 部分和。



下面展示的是这部分计算的展开伪代码。在这里，最内层循环发生的计算次数已经从最开始的2次（一次乘法和一次加法）发展到了现在的128次（ $2 \times 4(m) \times 4(n) \times 4(k)$ ）。

</> Gemm-4*4-k维度展开伪代码

```

1  for (int m = 0; m < M; m += 4) {
2      for (int n = 0; n < N; n += 4) {
3          C[m + 0..3][n + 0..3] = 0;
4          C[m + 0..3][n + 0..3] = 0;
5          C[m + 0..3][n + 0..3] = 0;
6          C[m + 0..3][n + 0..3] = 0;
7          for (int k = 0; k < K; k += 4) {
8              C[m + 0..3][n + 0..3] += A[m + 0..3][k + 0] * B[k + 0][n + 0..3];
9              C[m + 0..3][n + 0..3] += A[m + 0..3][k + 1] * B[k + 1][n + 0..3];
10             C[m + 0..3][n + 0..3] += A[m + 0..3][k + 2] * B[k + 2][n + 0..3];
11             C[m + 0..3][n + 0..3] += A[m + 0..3][k + 3] * B[k + 3][n + 0..3];
12         }
13     }

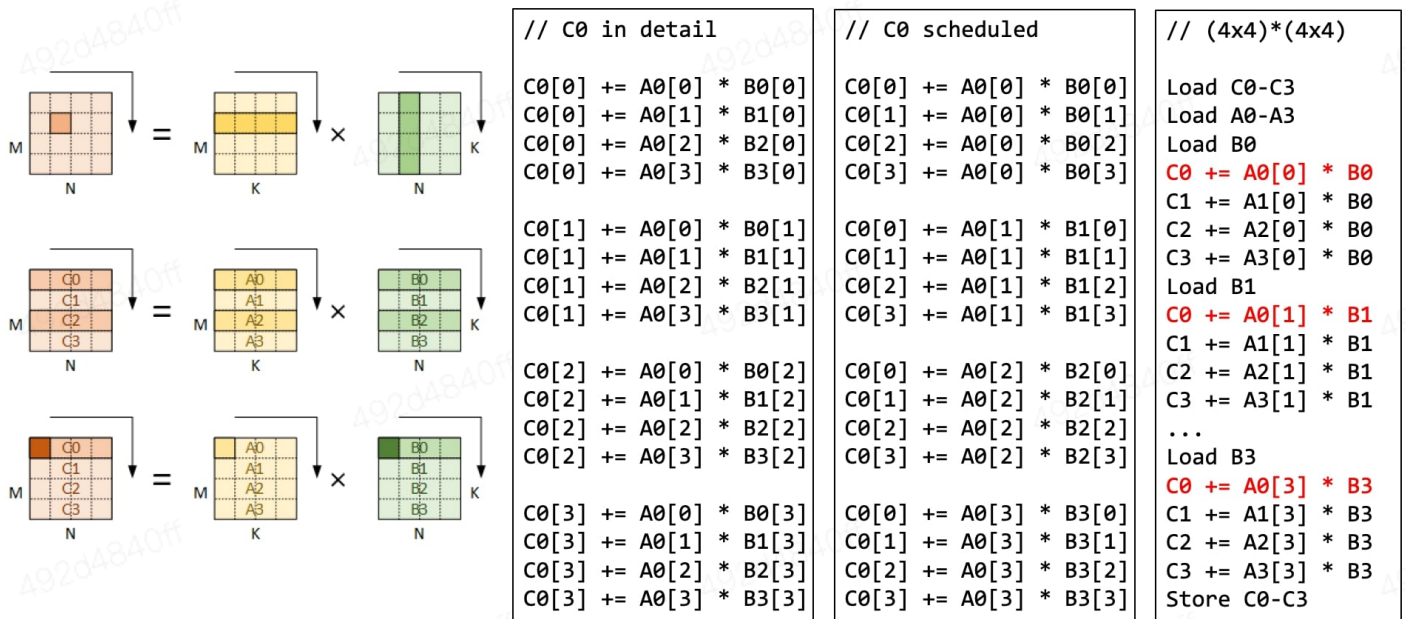
```

在对N和M维度进行展开时，我们可以分别复用A和B的数据。在对K展开时，其局部使用的C矩阵数据 ($C[m + 0..3][n + 0..3]$) 是一致的，那么在K迭代时可以将部分和累加在寄存器中——最内层循环结束后再一次性写到C的内存中，这样一来，在k维度的计算时对C数据的访问就变成了1次。而总的内存访问次数则变为了 $MN + (1/4 + 1/4)MNK \sim 1/2MNK$ (MN相比于MNK可忽略)，可以看到最后得到的加速比为8X。

实际上，现代处理器的SIMD能力远超示例中展开成4的情况(视处理器的向量寄存器位宽和向量计算单元而定)，因此可以使用更为激进的计算核，从而得到更为优化的性能，但究其原理仍然和本例相似。

到目前为止，我们已经对整体计算进行了三次维度拆分、展开并且利用寄存器实现访存复用优化。这对最基础版本的计算似乎已经足够了，因为数百条稳定的顺序计算指令对处理器流水线已经很友好，而且编译器可以帮助我们做好软件流水这样的指令调度。然而一条计算指令只能完成一次乘加操作 (MLA) 还是没有充分利用现代处理器的计算能力。实际上目前即使是最低端的移动手机处理器都会带有SIMD支持，因此前文提到的访存和计算操作都可以被向量化。下面，我们将再进一步，利用向量操作来提高整体计算的性能。

(4x4)x(4x4) Vector load/store/arithmetic



向量化4x4计算核

上面左侧展示的三幅小图分别展示了两个4x4矩阵相乘向量化的要素：首先是计算一个输出元素使用的输入元素；然后是对各个矩阵内存的编码，均以行优先的形式编号；最后是向量化的具体计算方法。

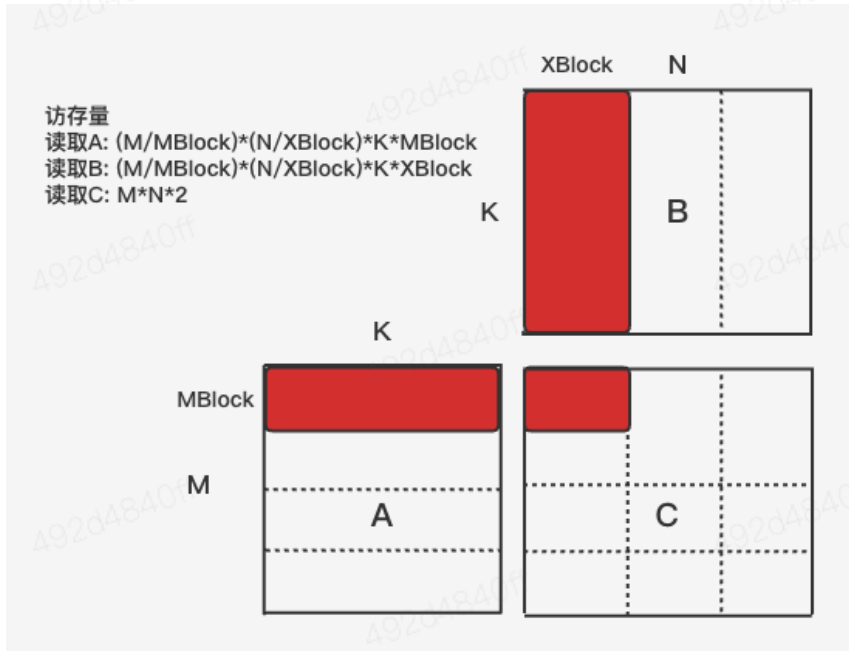
这里的向量编号方式假定输入输出的内存布局都是行优先，那么两个输入各自的16个元素通过4次向量访存（当前移动端处理器提供的向量寄存器通常为128bits）即可加载到寄存器中。由矩阵乘的外积算法可知，输入B中的行可一次性用作输出的计算，而输入A的行则要拆分使用。

上面右侧列出了三份伪代码。其中，第一份 *C0 in detail* 是计算C0中四个输出元素的朴素方法的展开，连续四次计算得到一个C0元素的结果。将计算过程稍作重排，即可得到 *C0 scheduled* 展示的计算，这里连续的四次计算分别处理了C0中四个元素的1/4结果，在连续的四次计算中，重排前只有对A的访存是连续的，重排后B和C的访存都是连续的。那么向量化这些访存和计算，即可得到第三列伪代码中红色C0部分。而第三列是通过通过对 C1、C2、C3 进行类似的处理得到的。

施行向量化操作后，原本需要64条计算指令的计算过程所需指令减少到16条，访存也有类似效果。而向量化对处理器资源的高效使用，又带来了进一步优化空间，例如可以一次计算8×8个局部输出。

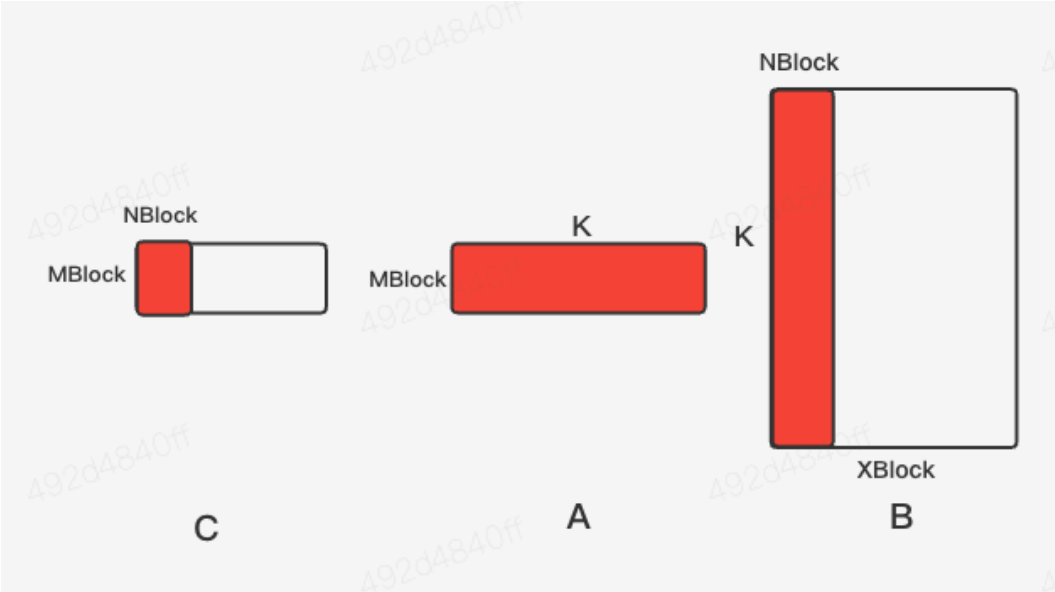
矩阵分块策略

为了减少Gemm计算时的访存开销，最有效的方法是对Gemm的计算进行分块，并将分块之后的数据保存在CPU的Cache中，充分利用Cache提高访存效率。如下图所示，将A矩阵按照MBlock进行行分块，将B矩阵按照XBlock进行列分块，计算核大小为MBlock x NBlock。计算时将需要用到的分块保存在CPU的各级Cache中，从而极大提高内存的访问效率，减少整体计算的内存开销。



由于A中的每个行块都要重复地和B中的每一个列块进行小分块的Gemm操作，为了高效利用Cache（Cache miss率尽量低），因此需要根据具体CPU的Cache结构特点（速度：L1D>L2>L3，容量 L1D<L2<L3）来分配Cache和数据块之间的存放关系。大致的内存分配策略如下：

- 尽可能将下图中红色部分（计算核对应的输入矩阵数据）保存在L1D中，以确保最内层循环中发生的计算不受访存带宽的限制（当K较大时，只能保证KBlock对应的计算发生在L1 Cache中）。
- 矩阵C的中间计算结果保存在寄存器中。
- 由于计算C的每一个行块都需要重复遍历一次整个B矩阵，因此将B的列块尽可能存放在L2中得每次读取B的一个列块都是从L2中读取，访存代价最小。当使用多进程来加速计算时，B的列块也可以被共享同一L2的多个进程所共享。

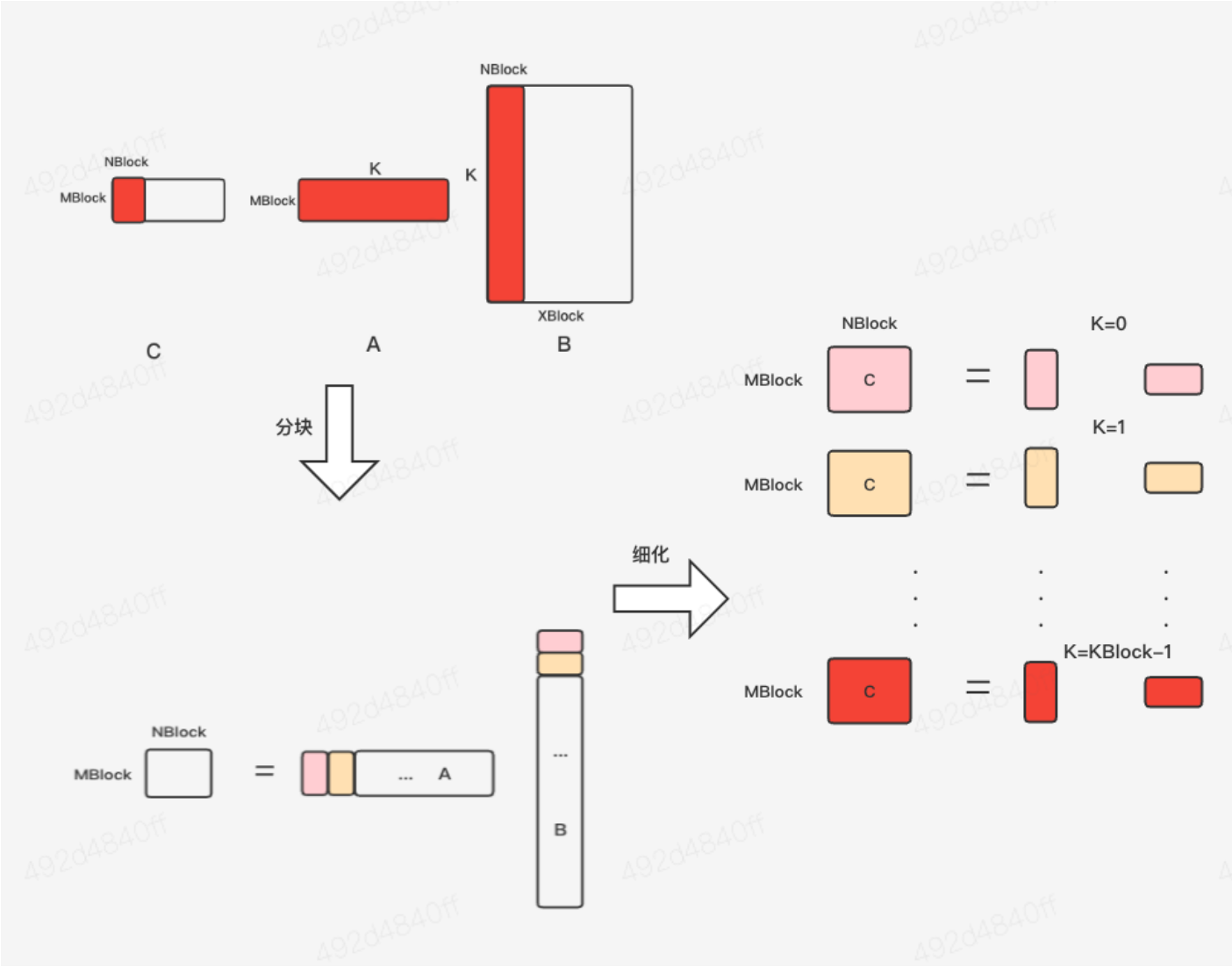


通过上面的分配策略，并结合具体CPU提供的硬件资源(寄存器数量，L1D和L2的大小)，可以确定不同部署环境（CPU）下Gemm计算中的最佳分块大小：

- 根据CPU处理器的寄存器数量得到Mr和nr的具体大小，寄存器容量 > $MBlock * NBlock * sizeof(elem)$ 。
- 根据L1D Cache的大小结合MBlock和NBlock计算出KBlock， $KBlock = L1D / (MBlock + NBlock)$ 。
- 再根据L2的大小计算出B矩阵的列块大小XBlock， $XBlock = (L2 - L1D) / K$ 。（当前CPU使用的Cache是Inclusive的，且L2是指令缓存和数据缓存合并的）

根据以上计算得到XBlock、KBlock、MBlock和NBlock之后，便可以首先对输入矩阵的N、K维度分别进行XBlock和KBlock分块处理，然后在XBlock、KBlock的基础上再分别对A矩阵进行MBlock分块，对B矩阵进行NBlock分块处理（内存布局为行优先）。

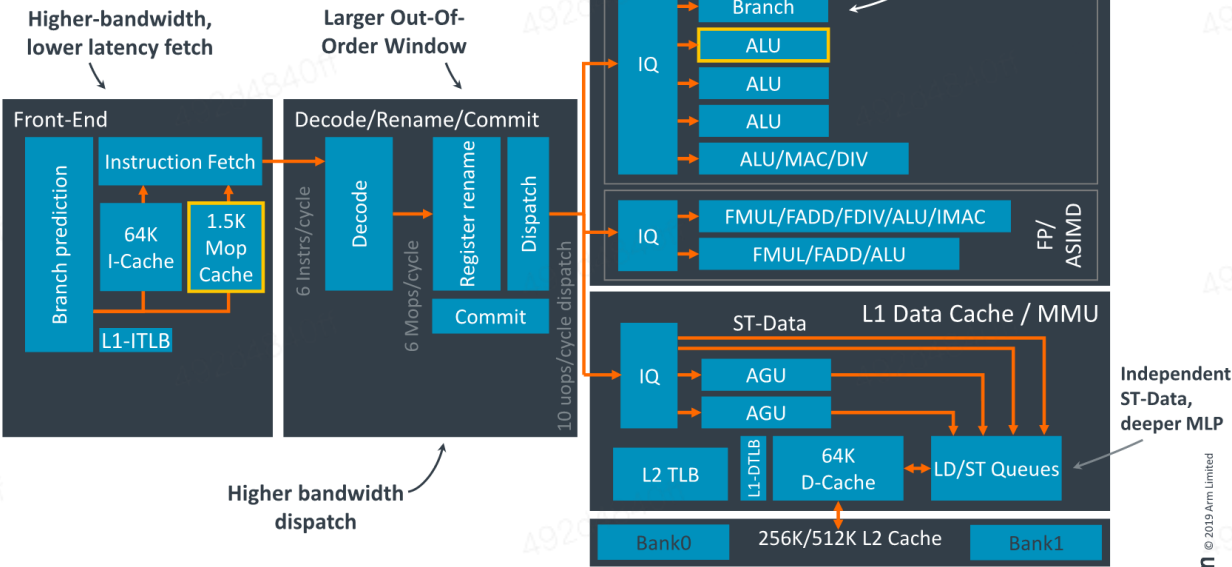
经过上述分块后，Gemm的计算核为 $A(MBlock * KBlock) \times B(KBlock * NBlock) \rightarrow C(MBlock * NBlock)$ ，其计算示意图如下：



计算时，在K维度上，会读取A中一列数据和B中一行数据进行矩阵外积算法，得到大小为MBlock*NBlock的C，如此循环K/KBlock次，完成最内层循环的计算。具体过程在 [Gemm-矩阵分块与寄存器优化](#) 已经详细介绍。

在该计算核的计算过程中，计算量为 $2MBlock * NBlock * KBlock$ ，访存量为 $(MBlock + NBlock) * KBlock + 2MBlock * NBlock$ ，接下来需要根据处理器信息来判断该计算能否隐藏数据的访存。以 ARM cortex A77为例进行分析，该芯片微架构图如下：

Cortex-A77: Microarchitecture overview



The embargo for this content presented at Arm Tech Day will lift on Sunday, May 26 at 9:00 p.m. PT. Corresponding UK and Taiwan times are: Monday, May 27 at 5:00 a.m. BST / Monday, May 27 at 12:00 p.m. China Standard Time

根据 A77的数据手册得到：

- FP32 SIMD Load throughput=2，即单周期可以load 8个float 数据。
- FP32 SIMD FMLA throughput=2，即单周期可以进行16次乘加运算。

Instruction characteristics

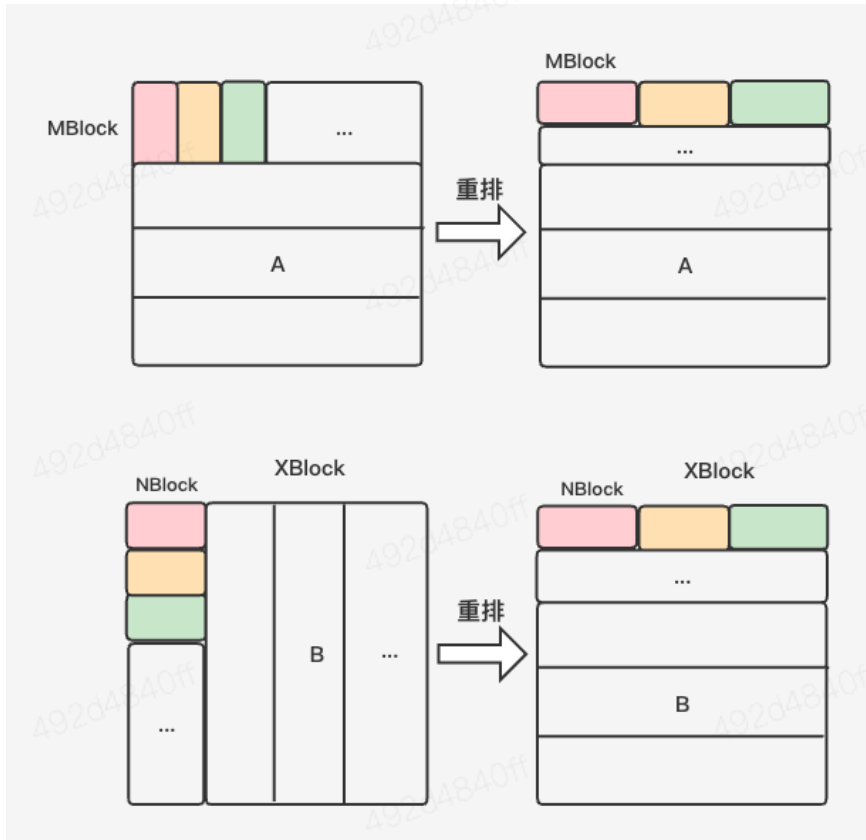
Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load vector reg, literal, S/D/Q forms	LDR	-	2	L	-
Load vector reg, unscaled immed	LDUR	5	2	L	-
Load vector reg, immed post-index	LDR	5	2	L, I	-
Load vector reg, immed pre-index	LDR	5	2	L, I	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP multiply accumulate	FMLA, FMLS	4 (2)	2	V	1

该处理器的计算访存比为2。因此可以得出结论，在计算核所需的A和B数据均在L1的前提下，若计算核的计算访存比高于这一数值，则计算不会因为数据的Load而被阻塞，能够发挥出处理器的最佳性能。

内存重排

虽然在对数据进行分块时，计划将A和B矩阵的红色部分数据置于L1D Cache中，但是在真正运行时数据却不一定都在L1D中，原因在于，B矩阵的列块在原来的大矩阵中内存并不连续（行优先存储），其次A中的列数据由于内存不连续，也不能使用SIMD进行load，因此需要考虑对A和B矩阵进行内存重排。



对矩阵A进行数据重排是将A中MBlock行数据的相同列拷贝到一起，对应图中将A重排的过程。B矩阵的数据重排操作是将B经过XBlock分块后的NBlock列数据拷贝到连续的内存地址中，它对应上图中将B重排的过程。

3.当前Lite实现方案

A矩阵分块与重排

当前Lite实现中，在PrepareForRun阶段，会调用ReInitWhenNeeded接口对weight数组进行内存重排操作。

```
266  #ifdef ENABLE_ARM_FP16
267  template <>
268  void GemmLikeConv<PRECISION(kFP16), PRECISION(kFP16)>::PrepareForRun() {
269      ReInitWhenNeeded();
270  }
```

```

34 template <PrecisionType Ptype, PrecisionType Otype>
35 class GemmLikeConv : public KernelLite<TARGET(kARM), Ptype> {
36 public:
37     GemmLikeConv() = default;
38     ~GemmLikeConv() {}
39
40     virtual void ReInitWhenNeeded() {

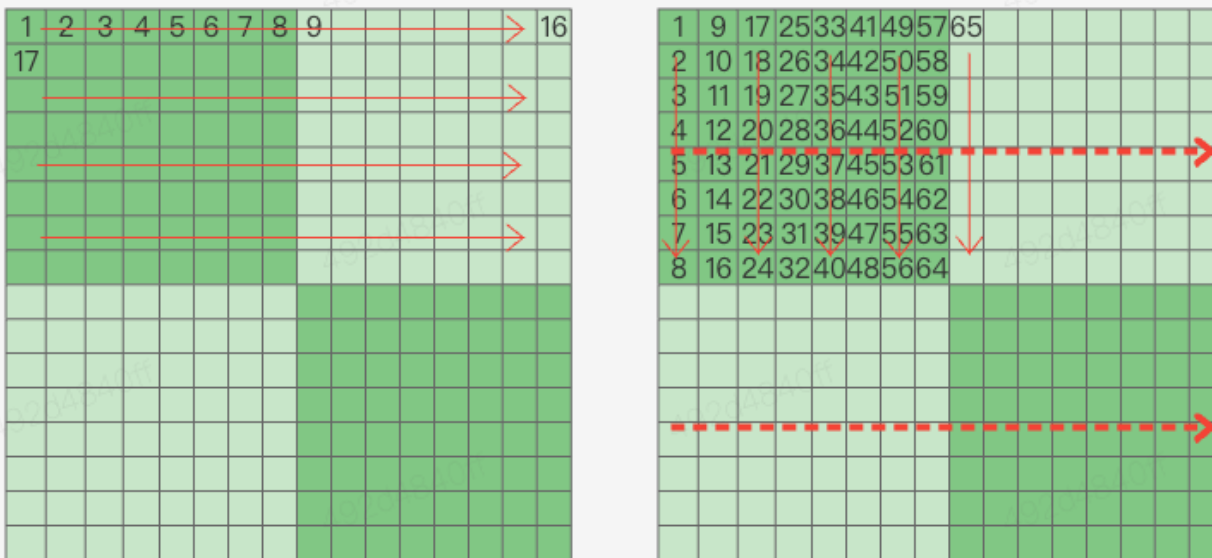
```

```

79     //! select conv gemmlike kernel
80 > if (kw == 1 && sw == 1 && pw == 0 && kps_equal && pads_equal) {...
83 > } else {...
87     }
88     if (!flag_trans_weights_ && n > 1 && m > 1) {
89         if (param.filter->precision() == PrecisionType::kFP16) {
90 #ifdef ENABLE_ARM_FP16
91         lite::arm::math::fp16::trans_gemm_weights_fp16(
92             *(param.filter), weights_, param.groups, &ctx);
93 #else

```

具体的重排操作在arm::math::fp16::repackA_8x16中执行，它的大致原理（为了方便展示，此处以8x8的计算核为例）如下图所示：



对矩阵A而言，计算核的方向（即K维度）的方向是水平方向，每次取完水平方向的1个元素后，会跳到下一行继续取用相同列的数据，重复MBlock次。

- 从整个L1 Cache的角度看，一次计算核所需A的内存块大小为8*K，以K=1024，且单个元素为2Byte为例，则大小为8*1024*2/1024=16KB，并没有超过大部分主流移动端处理器的L1D Cache大小（64KB）。
- 从Cache Line的角度看，即使是上图左侧的内存排布方式，一次load到Cache Line中的32个值（Cache Line Size = 64B，DataType=FP16）中的后31个值虽然不会在下一次load前就马上被用掉，但也就隔7次load后就会被用到，所以不会出现L1 Cache miss。

B矩阵分块与重排

对于矩阵B（即输入特征图矩阵）而言，首先使用`ctx->llc_size()`接口获取last level size，即L3 size。

```
1827 size_t llc_size = ctx->llc_size() > 0 ? ctx->llc_size() : 512 * 1024;
```

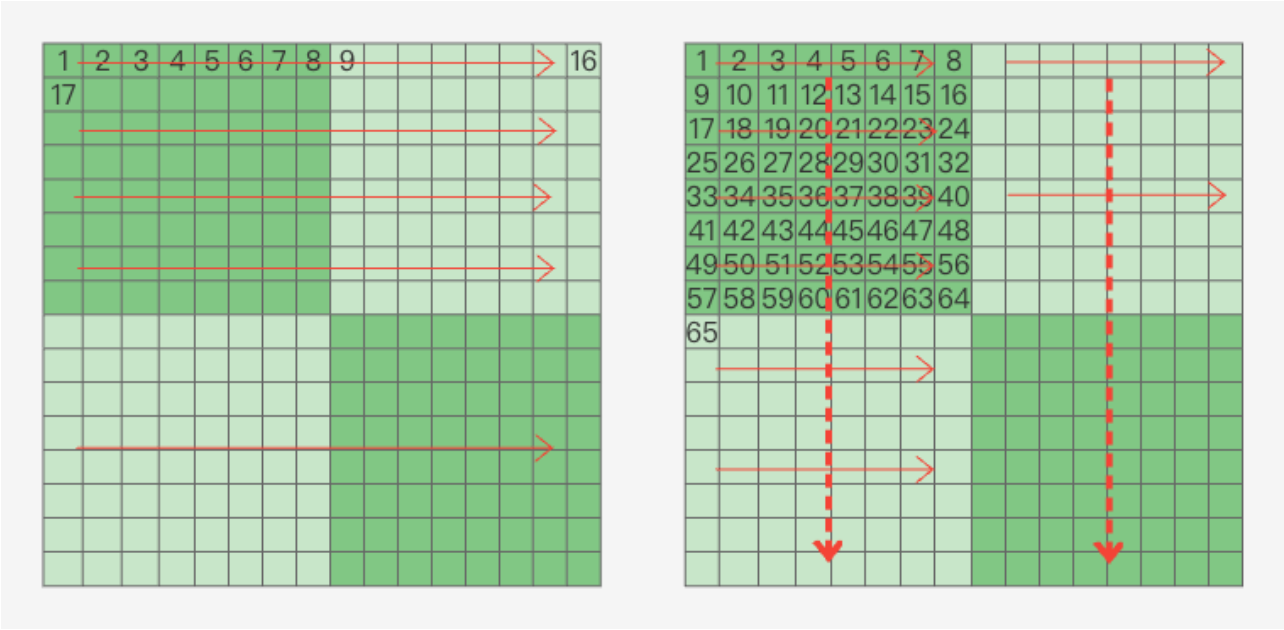
在进行计算前，通过上节介绍的分块策略计算出XBlock。使B的XBlock块尽可能存放在L2中，访存代价最小。

```
1848 //! MBLOCK * x (result) + MBLOCK * k (A) + x * k (B) = l2
1849 X_BLOCK_COMPUTE_FP16(llc_size, MBLOCK_FP16, NBLOCK_FP16, KBLOCK_FP16, beta)
```

接着进行Gemm计算：

```
1855 //! apanel is pre_compute outside gemm
1856 for (unsigned int x0 = 0; x0 < N; x0 += x_block) {
1857     unsigned int xmax = x0 + x_block;
1858     if (xmax > N) {
1859         xmax = N;
1860     }
1861     int bblocks = (xmax - x0 + NBLOCK_FP16 - 1) / NBLOCK_FP16;
1862     remain = xmax - x0 - (bblocks - 1) * NBLOCK_FP16;
1863     if (remain > 0 && remain != 16) { ...
1864     }
1865     //! load bpanel
1866     float16_t *b_pannel = workspace;
1867     if (is_transB) { ...
1868     } else {
1869         loadb(b_pannel, B, ldb, 0, K, x0, xmax);
1870     }
1871     LITE_PARALLEL_COMMON_BEGIN(y, tid, M, 0, MBLOCK_FP16) {
1872         unsigned int ymax = y + MBLOCK_FP16;
1873         if (ymax > M) { ...
1874         }
1875         float16_t bias_local[8] = {0};
1876         if (has_bias) { ...
1877         }
1878         float16x8_t vbias = vld1q_f16(bias_local);
1879         // prepare out data
1880         GEMM_PREPARE_C(float16_t, NBLOCK_FP16)
1881         const float16_t *a_ptr_l = A_packed + y * K;
1882         const float16_t *b_ptr = b_pannel;
1883         for (int xb = 0; xb < bblocks; xb++) { ...
```

对于B的每一个XBlock列块，将其依次和A的每个MBlock行块进行Gemm运算，但在计算之前，会首先按照计算出来的NBlock对XBlock块进行内存重排操作（loadb函数），尽量减少cache miss率。具体的重排操作原理如下图所示：

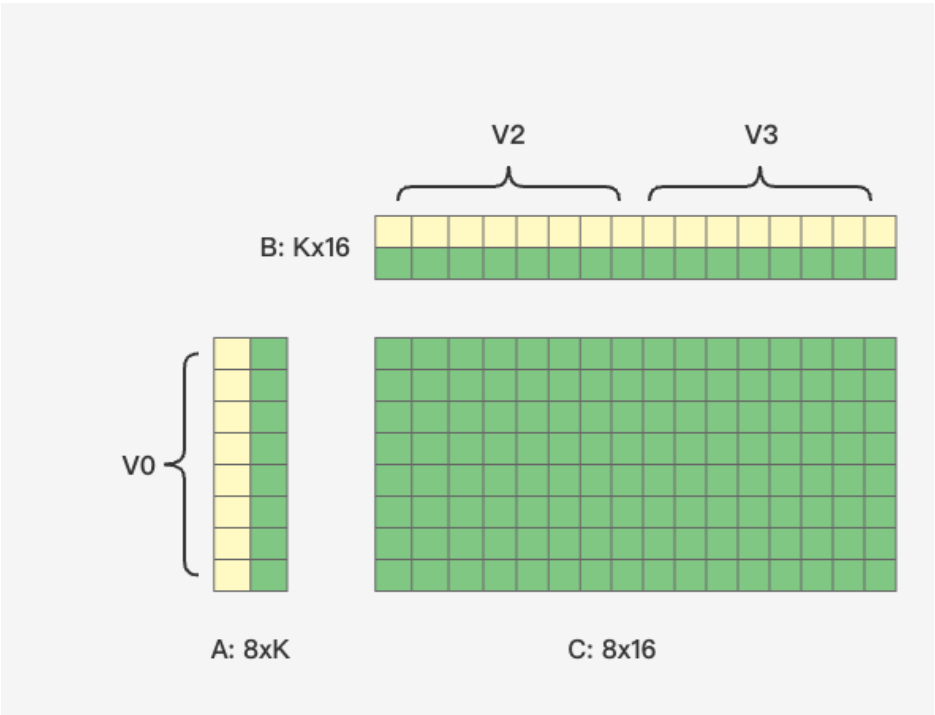


依然是行优先存储，计算核大小为8 x 8，计算方向是K方向，即B矩阵的垂直方向，每次循环加载8个元素（128/16=8）后，就需要跳到下一行去加载8个元素。假设Cache Line Size 为64Byte，数据元素大小为2byte，则执行一次访存操作会将32个元素读到L1D Cache中。

- B矩阵普通的内存排布顺序就如左侧红色箭头所示。每次访存load到Cache Line中实际只需要前8个值，后24个值暂时不会用。当B的行数K小的时候，K循环结束时L1 Cache还没被填满，则每个Cache Line的后24个值还能命中。但当A的列数足够大时，后面的值还没被用Cache就满了，最早load的Cache Line会被覆盖，当你真正要用后24个值的时候还得重新load，此时就发生了L1 Cache miss。
- 而采用右侧这样的内存排布，计算核依次加载的内存是连续的，一次load到Cache Line中的32个值会在连续的4次运算中被用到，这就减少了Cache miss，从而提高了性能。

8x16计算核

针对FP16数据类型，当前Lite采取的计算核大小为8x16。



由于针对A矩阵进行了内存重排操作，同一列的MBlock个数据在内存中的地址是连续的，因此在计算核的取数过程中，仅需要3条ldr指令就可满足 $8*16*2=512$ 次乘加操作所需的操作数，并得到1/K的矩阵C中间累加值。同时，Lite还利用了向量化来最大化利用CPU的SIMD能力，这512次乘加操作可以转换成 $8*2=16$ 条FMLA指令，从数据上来看，完全可以通过手动重排汇编指令(FMLA指令)来掩藏ldr指令的访存延迟（Exec Latency = 5）。

参考链接

<https://github.com/flame/how-to-optimize-gemm/wiki>

<https://developer.arm.com/documentation/101111/0101/?lang=en>

<https://developer.arm.com/documentation/swog011050/c/>

<https://jackwish.net/blog/2019/gemm-optimization.html>

<https://no5-aaron-wu.github.io/2021/12/09/AI-Algorithm-12-GEMM/>